COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 031 - VISUALIZATIONS AND DATA ANALYSIS	**SECTION:**	CPE21S1
DATE PERFORMED:	Sep 12, 2026	**DATE SUBMITTED:**	Sep 12, 2026
NAME:	Mendoza, Roy Gabriel Mendoza, Ryo Shin Obias, Jave Joaquin	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 7 **[Purpose Use of Color]**

1. PROCEDURE OUTPUT

CODE

```
import matplotlib.pyplot as plt
import numpy as np
import colorsys

# Function to convert HSL (Hue, Saturation, Luminance) to RGB
def hsl_to_rgb(h, s, l):
    return colorsys.hls_to_rgb(h, l, s)

# Create a figure with 3 subplots side-by-side
fig, axes = plt.subplots(1, 3, figsize=(15, 5)) # 1,3 instead of the 2,2 to show it all in a horizontal way

# ----------------------------
# 1. Hue demonstration
# ----------------------------
# Varying hue from 0 to 1, keeping saturation and luminance constant
hues = np.linspace(0, 1, 6, endpoint=False)
colors_hue = [hsl_to_rgb(h, 1, 0.5) for h in hues]

axes[0].bar(range(len(hues)), [1]*len(hues), color=colors_hue)
axes[0].set_title("Hue (Different Colors)")
axes[0].axis("off")

```

```python
# ----------------------------
# 2. Saturation demonstration (Now using GREEN)
# ----------------------------
# Keeping hue constant at 0.33 (Green), varying saturation
saturations = np.linspace(0.2, 1, 5)
colors_sat = [hsl_to_rgb(0.33, s, 0.5) for s in saturations]

axes[1].bar(range(len(saturations)), [1]*len(saturations), color=colors_sat)
axes[1].set_title("Saturation (Varying Intensity)")
axes[1].axis("off")

# ----------------------------
# 3. Luminance demonstration (Now using ORANGE)
# ----------------------------
# Keeping hue constant at 0.1 (Orange), keeping saturation constant, varying luminance
luminances = np.linspace(0.2, 0.8, 5)
colors_lum = [hsl_to_rgb(0.1, 1, l) for l in luminances]

axes[2].bar(range(len(luminances)), [1]*len(luminances), color=colors_lum)
axes[2].set_title("Luminance (Varying Brightness)")
axes[2].axis("off")

# Display the charts
plt.tight_layout()
plt.show()
```

OUTPUT

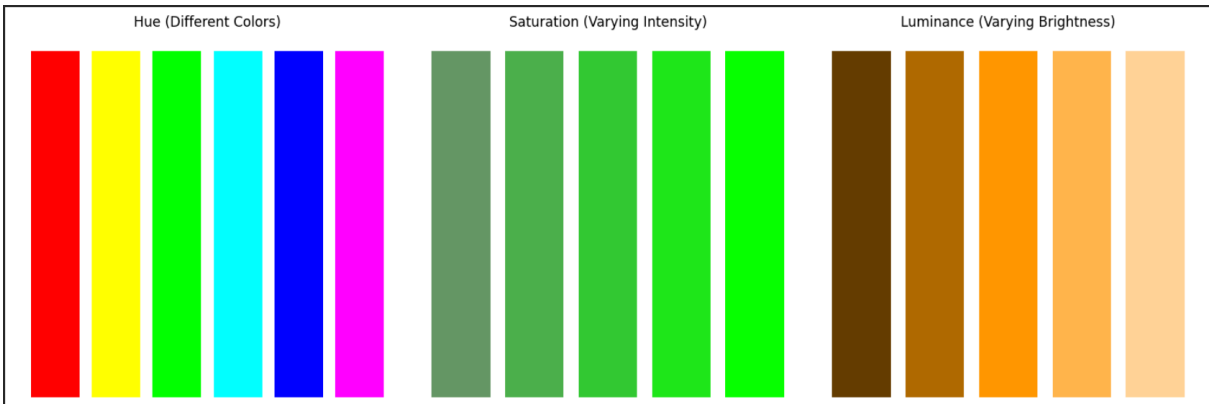


Table 7.1: Color and Perception

CODE

```python
#Code Here

#
import matplotlib.pyplot as plt
import numpy as np

# ----------------------------
# Sample Data
# ----------------------------

# Categorical data
categories = ["A", "B", "C", "D", "E"]
values = [10, 30, 20, 40, 25]

# Sequential data
sequential_values = np.arange(1, 11).reshape(2, 5)

# Diverging data
diverging_values = np.array([
    [-5, -4, -3, -2, -1],
    [ 0,  1,  2,  3,  4]
])

# Create figure
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

# ----------------------------
# 1. Categorical Color Scheme
# ----------------------------
axes[0].bar(
    categories,
    values,
    color=plt.cm.Set2.colors[:5]
)

axes[0].set_title("Categorical Color Scheme")
axes[0].set_xlabel("Categories")
axes[0].set_ylabel("Values")

# ----------------------------
# 2. Sequential Color Scheme
# ----------------------------
seq = axes[1].imshow(
    sequential_values,
    cmap="Blues"
)

axes[1].set_title("Sequential Color Scheme")
axes[1].set_xlabel("Increasing Values")
plt.colorbar(seq, ax=axes[1])

# ----------------------------
# 3. Diverging Color Scheme
# ----------------------------
div = axes[2].imshow(
    diverging_values,
    cmap="RdBu",
    vmin=-5,
    vmax=4
)

axes[2].set_title("Diverging Color Scheme")
axes[2].set_xlabel("Negative ← 0 → Positive")
plt.colorbar(div, ax=axes[2])

# Display the plots
plt.tight_layout()
plt.show()

```

OUTPUT

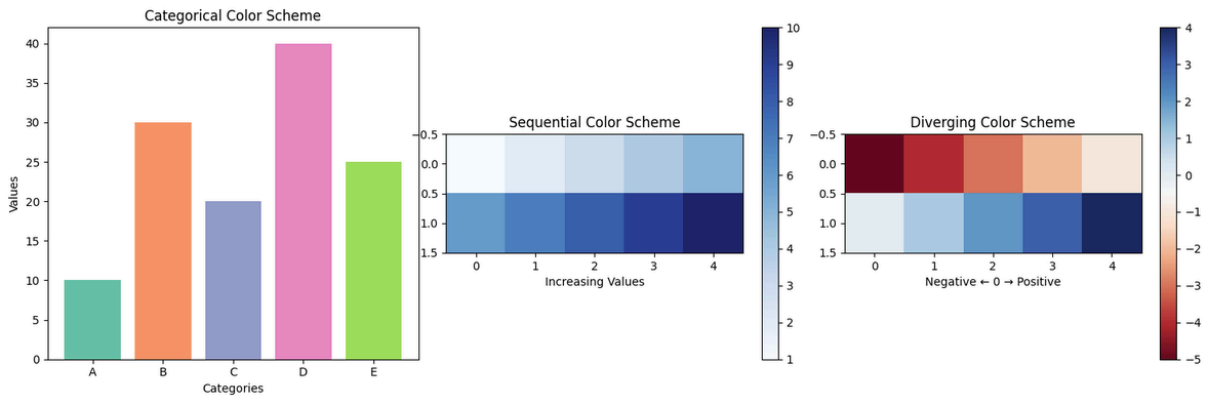


Table 7.2: Color Schemes and Types of Data

CODE

```
# Code here
import matplotlib.pyplot as plt
import colorsys

# Fixed saturation and luminance
saturation = 1.0
luminance = 0.5

# 5 different hues
hues = [0, 0.2, 0.4, 0.6, 0.8]

# Convert HSL colors to RGB
colors = [colorsys.hls_to_rgb(h, luminance, saturation) for h in hues]

# Print RGB values
for i, color in enumerate(colors):
    print(f"Color {i + 1} RGB value: {color}")

# Create pie chart
labels = ["Color 1", "Color 2", "Color 3", "Color 4", "Color 5"]

plt.pie(values, labels=labels, colors=colors, autopct="%1.1f%%")
plt.title("Custom Colors Using HSL")
plt.show()
```

OUTPUT

```
Color 1 RGB value: (1.0, 0.0, 0.0)
Color 2 RGB value: (0.7999999999999998, 1.0, 0.0)
Color 3 RGB value: (0.0, 1.0, 0.4000000000000024)
Color 4 RGB value: (0.0, 0.3999999999999999, 1.0)
Color 5 RGB value: (0.7999999999999998, 0.0, 1.0)
```

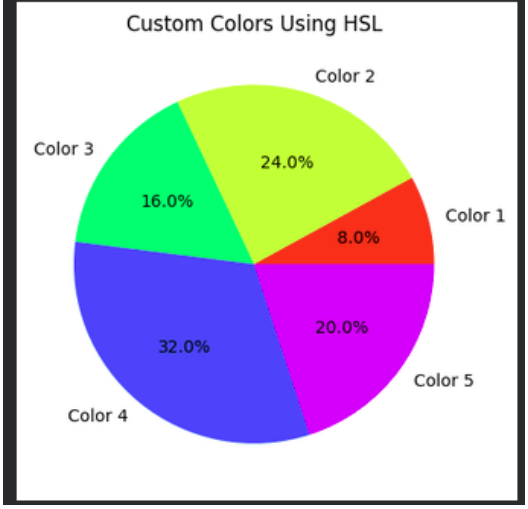


Table 7.3: Custom Color Using the HSL Color System

CODE

```python
#Code here
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# Sample data
categories = ["A", "B", "C", "D"]
values = [10, 25, 17, 30]

fig, axs = plt.subplots(3, 2, figsize=(12, 10))

# 1. Unnecessary Color (Bad)
axs[0,0].bar(categories, values, color=["orange", "cyan", "pink", "lime"])
axs[0,0].set_title("Bad: Unnecessary Colors")

# Good: One meaningful color
axs[0,1].bar(categories, values, color="teal")
axs[0,1].set_title("Good: Single Meaningful Color")

# 2. Excessive Color (Bad)
colors = sns.color_palette("husl", 12)
axs[1,0].bar(range(12), np.random.randint(5,20,12), color=colors)
axs[1,0].set_title("Bad: Excessive Colors")

# Good: Limited and consistent palette
sns.barplot(x=categories, y=values, palette="pastel", ax=axs[1,1])
axs[1,1].set_title("Good: Limited Palette")

# 3. Insufficient Contrast (Bad)
axs[2,0].bar(categories, values, color="lightyellow")
axs[2,0].set_facecolor("white")
axs[2,0].set_title("Bad: Low Contrast")

# Good: High Contrast
axs[2,1].bar(categories, values, color="navy")
axs[2,1].set_facecolor("white")
axs[2,1].set_title("Good: High Contrast")

# Show results
plt.tight_layout()
plt.show()
```

OUTPUT

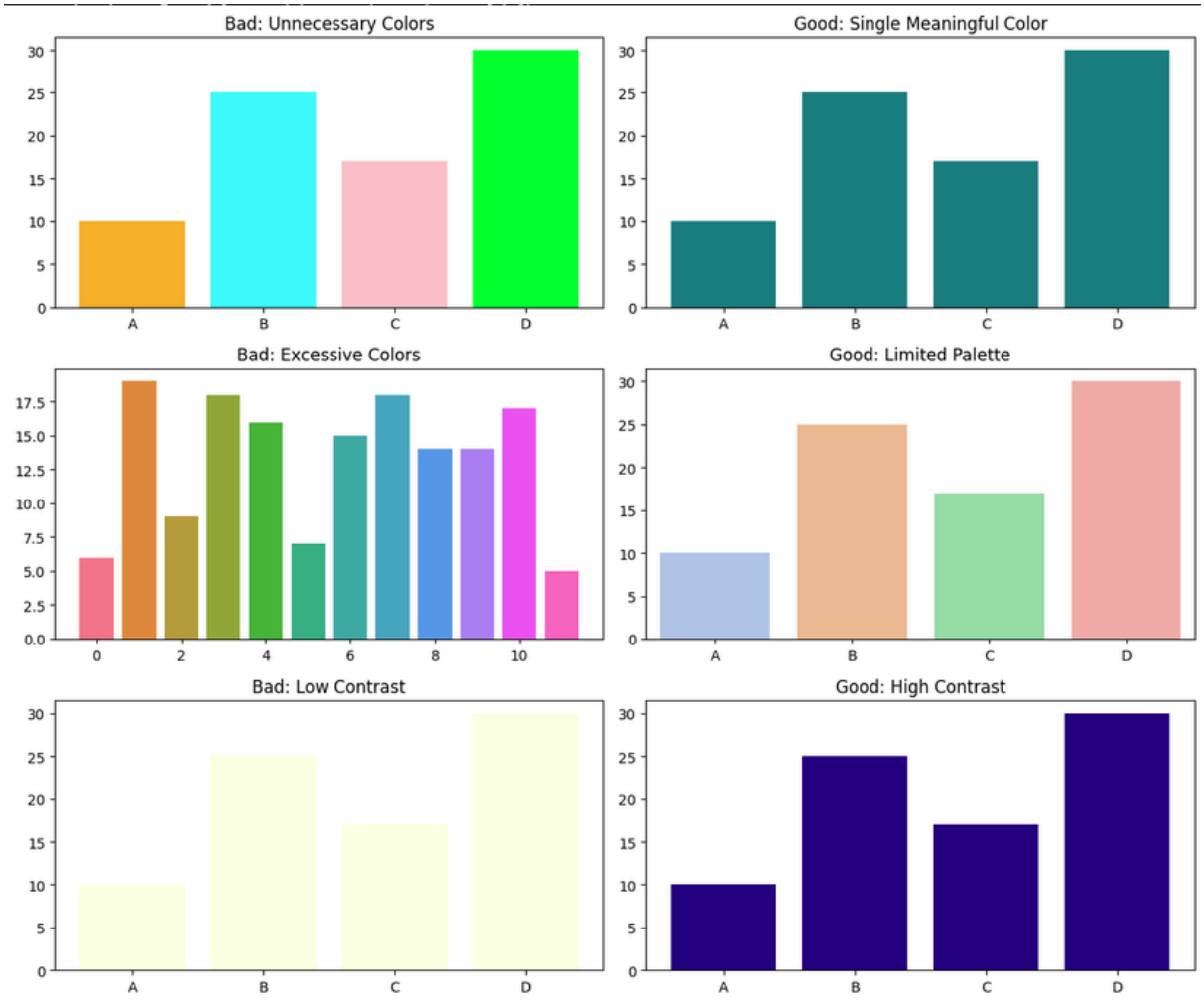


Table 7.4: Common Mistakes in the Use of Color in Data Visualization

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

QUESTION:

What are the top 10 highest-rated Roblox games, and how can we create a bar graph displaying their ratings on the x-axis and their visit counts on the y-axis?

CODE

Create a dataset

```python
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('roblox_games.csv', encoding='latin1')

print(df.columns.tolist())
```

```
['Rank', 'Name', 'Active', 'Visits', 'Favourites', 'Likes', 'Dislikes', 'Rating']
```

```python
# Sorting by Rating
df = df.sort_values('Rating', ascending=False)

# Selecting the top 10 highest-rated games
df = df.head(10)

# Selecting the columns
df = df[
    ['Name',
     'Rating',
     'Visits']
]

# Display the top 10 highest-rated games
df
```

Color and Perception

```python
import matplotlib.pyplot as plt
import colorsys
import numpy as np

# Get the top 10 highest-rated games
top_10_rated = df.sort_values('Rating', ascending=False).head(10).copy()

# Create ranking from 1 to 10
top_10_rated = top_10_rated.reset_index(drop=True)
top_10_rated['Rating Rank'] = top_10_rated.index + 1

# Clean the Visits column
top_10_rated['Visits'] = (
    top_10_rated['Visits']
    .astype(str)
    .str.replace(',', '')
    .astype(float)
)

# Get ranking and visits
ranking = top_10_rated['Rating Rank']
visits = top_10_rated['Visits']

# Create the colors

# 1. Varying Hue
hues = np.linspace(0, 1, 10, endpoint=False)
colors_hue = [
    colorsys.hls_to_rgb(h, 0.5, 1)
    for h in hues
]

# 2. Varying Saturation
saturations = np.linspace(0.3, 1, 10)
colors_sat = [
    colorsys.hls_to_rgb(0.6, 0.5, s)
    for s in saturations
]

# 3. Varying Luminance
luminances = np.linspace(0.3, 0.8, 10)
colors_lum = [
    colorsys.hls_to_rgb(0.6, l, 1)
    for l in luminances
]

```

```
# Create three bar graphs
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

# Varying Hue
axes[0].bar(ranking, visits, color=colors_hue)
axes[0].set_title("Varying Hue")
axes[0].set_xlabel("Ranking based on their Ratings")
axes[0].set_ylabel("Number of Visits")

# Varying Saturation
axes[1].bar(ranking, visits, color=colors_sat)
axes[1].set_title("Varying Saturation")
axes[1].set_xlabel("Ranking based on their Ratings")
axes[1].set_ylabel("Number of Visits")

# Varying Luminance
axes[2].bar(ranking, visits, color=colors_lum)
axes[2].set_title("Varying Luminance")
axes[2].set_xlabel("Ranking based on their Ratings")
axes[2].set_ylabel("Number of Visits")

# Show the graphs
plt.tight_layout()
plt.show()
```

```python
import matplotlib.pyplot as plt
import numpy as np
import matplotlib.colors as mcolors

# Get the top 10 highest-rated games
top_10_rated = df.sort_values('Rating', ascending=False).head(10).copy()

# Create ranking from 1 to 10
top_10_rated = top_10_rated.reset_index(drop=True)
top_10_rated['Rating Rank'] = top_10_rated.index + 1

# Clean the Visits column
top_10_rated['Visits'] = (
    top_10_rated['Visits']
    .astype(str)
    .str.replace(',', '')
    .astype(float)
)

# Get ranking and visits
ranking = top_10_rated['Rating Rank']
visits = top_10_rated['Visits']

# Calculate average visits
average_visits = visits.mean()

# Create three bar graphs
fig, axes = plt.subplots(1, 3, figsize=(18, 6)) # Increased figsize for better readability

# 1. Categorical Color Scheme
# Using tab10 for 10 distinct categories, as Set2 only has 8.
axes[0].bar(
    ranking,
    visits,
    color=plt.cm.tab10.colors[:len(ranking)] # Use tab10 for 10 distinct colors
)
axes[0].set_title("Categorical Color Scheme")
axes[0].set_xlabel("Ranking based on their Ratings")
axes[0].set_ylabel("Number of Visits")


# 2. Sequential Color Scheme
# Define the colormap
cmap_seq = plt.cm.Blues
# Define the normalization based on the visits data
norm_seq = mcolors.Normalize(vmin=visits.min(), vmax=visits.max())
# Create a ScalarMappable object
sm_seq = plt.cm.ScalarMappable(cmap=cmap_seq, norm=norm_seq)
sm_seq.set_array(visits) # Set the data for the mappable

```

```
colors_seq_mapped = sm_seq.to_rgba(visits)

axes[1].bar(
    ranking,
    visits,
    color=colors_seq_mapped # Use the mapped colors
)
fig.colorbar(sm_seq, ax=axes[1], orientation='vertical', label='Number of Visits') # Add colorbar
axes[1].set_title("Sequential Color Scheme")
axes[1].set_xlabel("Ranking based on their Ratings")
axes[1].set_ylabel("Number of Visits")


# 3. Diverging Color Scheme
# Define the colormap
cmap_div = plt.cm.RdBu
# Define the normalization centered at average_visits
# Calculate the maximum absolute difference from the average to set symmetric vmin/vmax
max_diff = max(abs(visits.min() - average_visits), abs(visits.max() - average_visits))
vmin_div = average_visits - max_diff
vmax_div = average_visits + max_diff

norm_div = mcolors.TwoSlopeNorm(vmin=vmin_div, vcenter=average_visits, vmax=vmax_div)
# Create a ScalarMappable object
sm_div = plt.cm.ScalarMappable(cmap=cmap_div, norm=norm_div)
sm_div.set_array(visits) # Set the data for the mappable

# Get colors for the bars from the mappable
colors_div_mapped = sm_div.to_rgba(visits)

axes[2].bar(
    ranking,
    visits,
    color=colors_div_mapped # Use the mapped colors
)

axes[2].axhline(
    average_visits,
    linestyle="--",
    color='black', # Make it more visible
    label=f"Average Visits ({average_visits:,.0f})"
)

fig.colorbar(sm_div, ax=axes[2], orientation='vertical', label='Visits Relative to Average') # Add colorbar
axes[2].set_title("Diverging Color Scheme")
axes[2].set_xlabel("Ranking based on their ratings")
axes[2].set_ylabel("Visits")
axes[2].legend()

# Show the graphs
plt.tight_layout()
plt.show()
```

Custom Colors with HSL

```python
# 4. Custom Colors with HSL

import matplotlib.pyplot as plt
import colorsys

# Get the top 5 highest-rated games
top_5 = df.sort_values('Rating', ascending=False).head(5).copy()

# Clean the Visits column
top_5['Visits'] = (
    top_5['Visits']
    .astype(str)
    .str.replace(',', '')
    .astype(float)
)

# Remove unnecessary/special characters from game names
top_5['Name'] = top_5['Name'].str.replace(
    r'[^\x00-\x7F]+', '', regex=True
)

# Create 5 custom HSL colors
hues = [0, 0.2, 0.4, 0.6, 0.8]
saturation = 1.0
luminance = 0.5

# Convert HSL colors to RGB
colors = [
    colorsys.hls_to_rgb(h, luminance, saturation)
    for h in hues
]

# Print RGB values
for i, color in enumerate(colors):
    print(f"Color {i + 1} RGB value: {color}")

# Create pie chart
plt.figure(figsize=(8, 6))

plt.pie(
    top_5['Visits'],
    labels=top_5['Name'],
    colors=colors,
    autopct="%1.1f%%"
)

plt.title("Visits of Top 5 Highest-Rated Roblox Games")

plt.show()
```

Common Mistakes and Corrections

[11] ✓ 0s

```python
# 5. Common Mistakes and Corrections

import matplotlib.pyplot as plt

# Get the top 10 highest-rated games
top_10_rated = df.sort_values('Rating', ascending=False).head(10).copy()

# Create ranking from 1 to 10
top_10_rated = top_10_rated.reset_index(drop=True)
top_10_rated['Rating Rank'] = top_10_rated.index + 1

# Clean the Visits column
top_10_rated['Visits'] = (
    top_10_rated['Visits']
    .astype(str)
    .str.replace(',', '')
    .astype(float)
)

# Get ranking and visits
ranking = top_10_rated['Rating Rank']
visits = top_10_rated['Visits']

# Create two bar graphs
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# ----------------------------
# Bad Example
# ----------------------------
random_colors = [
    "red", "cyan", "yellow", "purple", "lime",
    "orange", "pink", "blue", "green", "brown"
]

axes[0].bar(
    ranking,
    visits,
    color=random_colors
)

axes[0].set_title("Bad: Too Many Random Colors")
axes[0].set_xlabel("Ranking")
axes[0].set_ylabel("Visits")

```

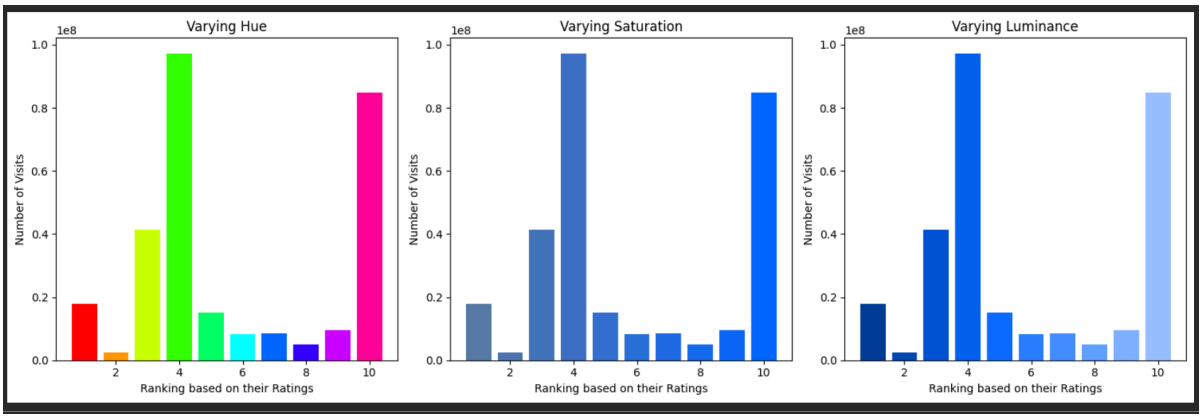
```
# ----------------------------
# Good Example
# ----------------------------
axes[1].bar(
    ranking,
    visits,
    color="darkblue"
)

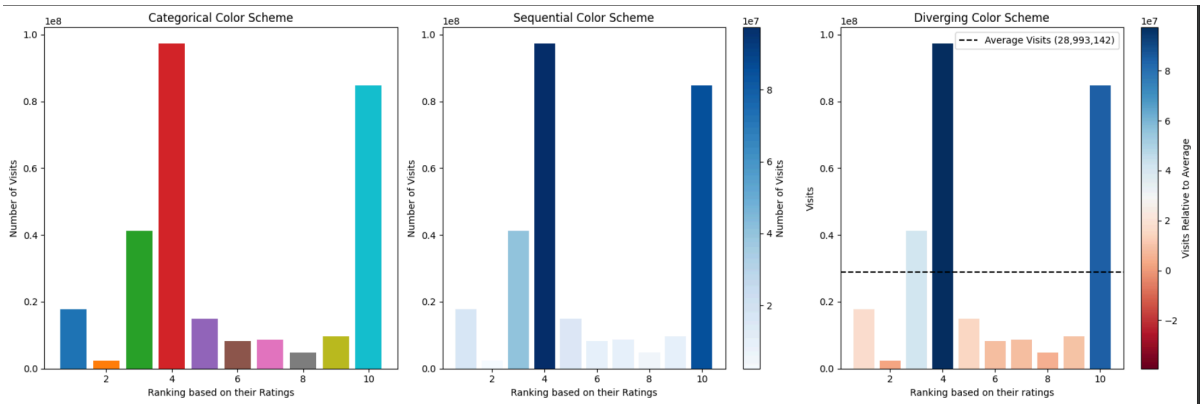
axes[1].set_title("Good: Consistent Color")
axes[1].set_xlabel("Ranking")
axes[1].set_ylabel("Visits")

# Show the graphs
plt.tight_layout()
plt.show()
```

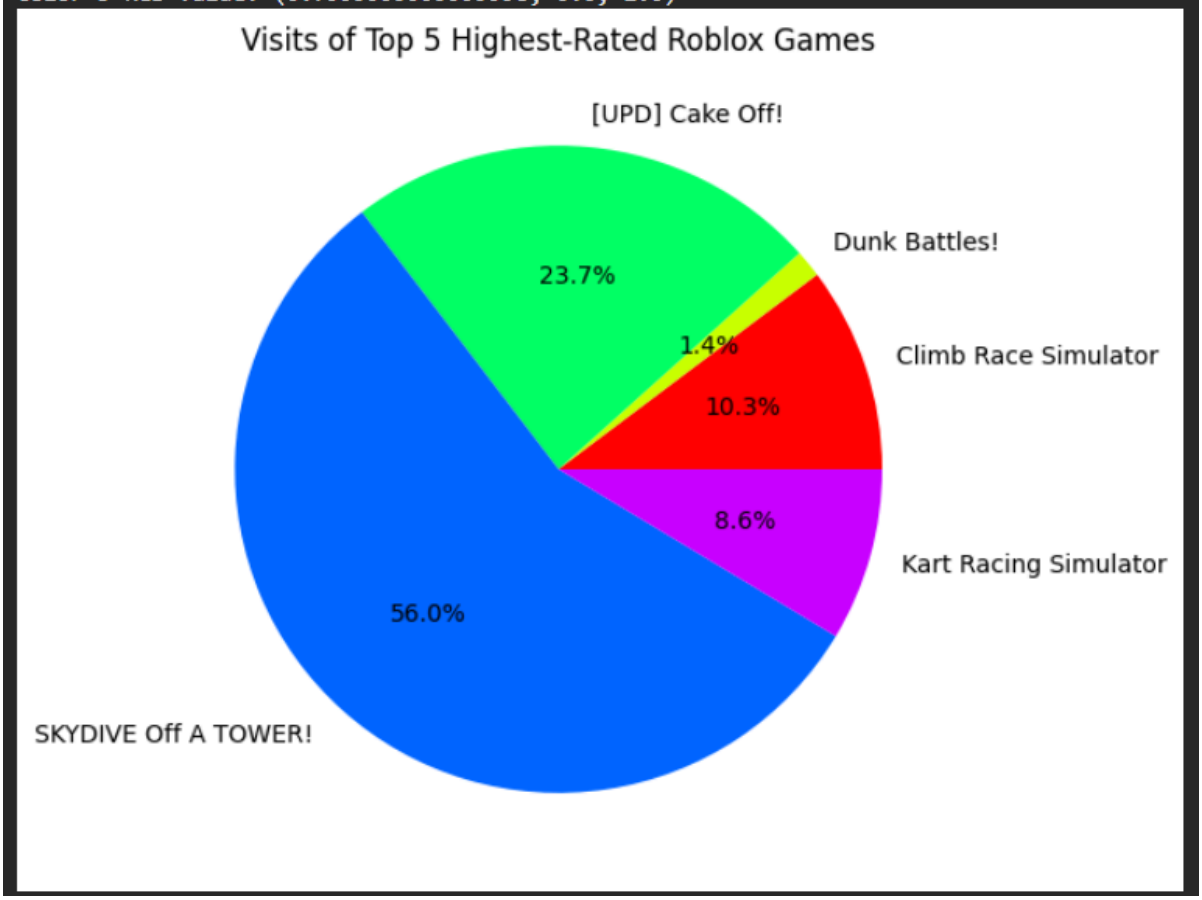
OUTPUT

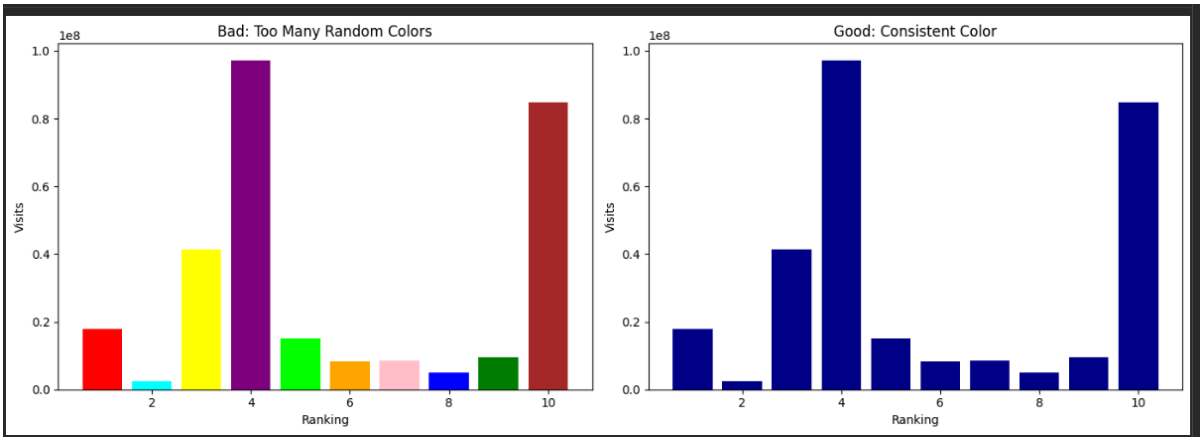
	Name	Rating	Visits
931	Climb Race Simulator	98.59	17,843,797
537	Dunk Battles!	98.53	2,383,311
532	[ð□□□UPD] Cake Off!	98.31	41,264,385
877	SKYDIVE Off A TOWER! ð□ª□	98.22	97,288,420
955	Kart Racing Simulator	98.07	14,985,646
595	Jule's RNG [ð□□□10K LUCK]	98.06	8,326,514
94	ð□□□ [UPD 14 HALLOWEEN] Anime Strike Simulator	98.02	8,605,990
845	Energy Drink Simulator â□¡	97.99	4,901,886
741	[ð□□□UPD]Demon Warriorsð□□º	97.96	9,594,169
345	[NEW FRUIT]Anime Dungeon Fighters	97.94	84,737,301


Varying Hue
Varying Saturation
Varying Luminance
Number of Visits
Ranking based on their Ratings
1e8


Categorical Color Scheme
Sequential Color Scheme
Diverging Color Scheme
Average Visits (28,993,142)
Number of Visits
Visits
Visits Relative to Average
Ranking based on their Ratings
Ranking based on their ratings
1e8
1e7

```
Color 1 RGB value: (1.0, 0.0, 0.0)
Color 2 RGB value: (0.7999999999999998, 1.0, 0.0)
Color 3 RGB value: (0.0, 1.0, 0.40000000000000024)
Color 4 RGB value: (0.0, 0.3999999999999999, 1.0)
Color 5 RGB value: (0.7999999999999998, 0.0, 1.0)
```





INDIVIDUAL ANALYSIS

Mendoza, Roy: The procedures of this activity has allowed us to explore the purpose of the color in data visualization. We began by understanding color perception, analyzing the hues, saturation and luminance and how they impact the interpretation of the data.Through all of these we were able to understand and know the common mistakes and errors in creating the data visualization.

And in the supplementary activity, we were able to apply all of these and more, like from what we have learned past couple of lessons ago, by utilizing a new Dataset , and demonstrate the learning by creating a bar chart with hue, saturation and luminance concepts. We were also able to create a basically like the good and bad data visualization techniques that is all about our dataset, which is basically the application of all of our learnings.

Mendoza, Ryo: In this activity, we analyzed the top 10 highest-rated Roblox games by sorting them according to their ratings and assigning a ranking from 1 to 10. We compared their number of visits using bar graphs, where the x-axis represents the ranking based on ratings and the y-axis represents the number of visits. This allowed us to easily compare the popularity of the highly rated games and see the differences in their visit counts.

We also applied different color and perception techniques, including hue, saturation, and luminance. We used categorical colors to distinguish rankings, sequential colors to show lower-to-higher visit values, and diverging colors to compare visits above and below the average. We also created a pie chart using custom HSL colors to show the proportion of visits among the top 5 highest-rated games.

Obias, Jave: In this supplementary activity, we practiced using hue, saturation, and brightness to understand their role in chart design, while also identifying the common mistakes that usually make visualizations hard to read. In the supplementary portion, we put these concepts into practice on a new dataset, applying both past lessons and recent visual techniques to construct refined bar charts. By building side-by-side examples of effective and poor visualization habits, we actively applied our learnings to create clearer, more practical data presentations.

INDIVIDUAL CONCLUSION

Mendoza, Roy: From this activity, I was able to properly assess and figure out the importance of purposely using and choosing the color in data visualization, because this can enhance the clarity of the leaders and avoid any misrepresentation. We have explored how different attributes like the hue, saturation, and luminance, and how they impact the perception of the viewer and how they can be used to convey the essence of the data easily. We also examined the different data types such as categorical, sequential and diverging, which required a specific color scheme for effective interpretation. Overall, effective color usage is not about making the visuals look good but to make them more appropriate, able to tell the viewers very easily about the data, or give insight to the audience without telling them explicitly.

Mendoza, Ryo: Overall, we learned that data visualization is not only about presenting numbers but also about presenting them clearly and meaningfully. Through the Roblox dataset, we were able to compare ratings, rankings, and visits while understanding how different color schemes affect the way data is interpreted. We also learned that using appropriate and consistent colors makes a visualization easier to understand, while too many

random colors can make the data confusing.

Obias, Jave: In conclusion, this activity demonstrated how strategic color choices directly impact data readability when analyzing top Roblox games. By applying categorical, sequential, and diverging color palettes to our bar charts, we were able to clearly separate rankings, highlight visit intensities, and easily spot games performing above or below the average. Breaking down the top five games with an HSL-tailored pie chart made proportional visits easy to digest, while contrasting cluttered visuals with unified color schemes proved that simpler design choices prevent distraction. Overall, learning to apply HSL principles effectively showed us that good visualization isn't just about making charts look nice, it's about making complex data instantly understandable and meaningful.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **None**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **None**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **None**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.



Please have the honor of pledge in the comment section or at the bottom of your activity document/exam or any form of file submission.

I affirm that I will not give or receive any unauthorized help on this activity/exam and that all work will be my own.